

BTS Services Informatiques aux Organisations (SIO)

Session 2026

Nom du candidat : SEILLER

Prénom du candidat : Julian

Parcours : SLAM

Fiche de réalisation professionnelle N°6

Intégration d'une solution de stockage hybride (PostgreSQL et Cloud MinIO/S3)

(Projet CardiacTS)

Période de réalisation : Du lundi 2 mars au vendredi 3 avril

Type de réalisation : En stage (Laboratoire TIMC)

Mode de réalisation : En équipe (Binôme)

Sommaire

Contexte Organisationnel.....	3
Problématique de gestion.....	3
Choix de la solution mise en œuvre.....	3
Réalisation de la solution.....	4
a - Prérequis et ressources à disposition.....	4
b - Mise en œuvre de la solution.....	4
c - Tests.....	4
i - Cas d'utilisation 1 : Téléversement (Upload) d'un fichier volumineux.....	4
ii - Cas d'utilisation 2 : Chargement et Visualisation via flux S3.....	5
iii - Cas d'utilisation 3 : Résilience et gestion des pannes réseau.....	5
Conclusions.....	5
Annexes.....	5
Compétences associées.....	6

Contexte Organisationnel

Cette réalisation a été effectuée lors de mon second stage au sein du Laboratoire TIMC (CNRS / Université Grenoble Alpes), situé à La Tronche. J'ai intégré l'équipe de recherche BCM (Biologie Computationnelle et Modélisation), sous la tutelle de Mr. Pierre Simon Jouk. J'ai travaillé en binôme avec un autre étudiant (Gabriel Da Silva Peixoto) sur la conception du projet "CardiacTS", une application destinée à moderniser la gestion des données de recherche médicale.

Problématique de gestion

Le projet CardiacTS doit traiter et stocker de **lourds fichiers d'imagerie médicale**. Historiquement, le laboratoire utilisait des fichiers dispersés (Excel et dossiers locaux), entraînant des risques de **corruption et de perte de données sensibles**. L'enregistrement de ces fichiers massifs (sous forme de BLOBs) directement dans une base de données relationnelle aurait causé de graves **problèmes de performances et de scalabilité**. Il fallait concevoir une architecture capable de gérer efficacement des **milliers de fichiers volumineux**, tout en assurant une **traçabilité stricte** exigée par le domaine médical.

Choix de la solution mise en œuvre

Pour résoudre ce problème de performances, nous avons opté pour une **gestion hybride du stockage** :

- **PostgreSQL** : Conservé uniquement pour gérer les **métadonnées légères** et les liens relationnels (informations patients, logs d'audit, utilisateurs).
- **MinIO (Compatible S3)** : Déployé comme solution de **stockage objet (Object Storage)** pour les fichiers d'imagerie massifs. MinIO offre les performances des grands clouds (grâce à l'**API standardisée Amazon S3**) tout en permettant d'être **auto-hébergé sur les serveurs du laboratoire**, garantissant ainsi que les données médicales confidentielles ne sortent pas de l'infrastructure du CNRS

Réalisation de la solution

a - Prérequis et ressources à disposition

- Serveur de base de données **PostgreSQL**.
- Serveur de stockage de données objets **MinIO**.
- Environnement **Java 21**, gestionnaire **Maven** et bibliothèque logicielle **SDK Java MinIO**.
- Dépôt de code partagé **GitHub** pour la collaboration en binôme.

b - Mise en œuvre de la solution

- **Configuration du client S3** : J'ai développé une classe de service dédiée aux interactions avec MinIO. Les identifiants de connexion (**AccessKey / SecretKey**) ont été strictement isolés et sécurisés via des **variables d'environnement (.env)**.
- **Logique Hybride** : Lors de l'ajout d'une image à un dossier patient, les métadonnées (nom, type de scan) sont insérées dans **PostgreSQL**, qui génère un identifiant unique (**UUID**). Cet UUID est ensuite utilisé par le code Java pour nommer et stocker le fichier physique dans un *bucket* **MinIO**.
- **Optimisation par Streaming** : Pour afficher une image haute résolution sans saturer la mémoire vive (RAM) de l'application, j'ai utilisé les **flux d'octets (InputStream)** de l'API S3. L'image est téléchargée et chargée en direct dans l'interface **JavaFX**.

c - Tests

i - Cas d'utilisation 1 : Téléversement (Upload) d'un fichier volumineux

- **Entrée / Action** : L'utilisateur sélectionne un cliché médical depuis l'interface JavaFX pour l'ajouter au dossier d'un patient.
- **Résultat attendu** : Le DAO enregistre la fiche descriptive dans **PostgreSQL**, puis le service S3 téléverse le fichier lourd dans le *bucket* **MinIO** avec succès. L'interface notifie l'utilisateur de la réussite.

ii - Cas d'utilisation 2 : Chargement et Visualisation via flux S3

- **Entrée / Action :** L'utilisateur clique sur une imagerie spécifique dans la liste d'un dossier patient.
- **Résultat attendu :** Le fichier est interrogé et récupéré depuis **MinIO via le protocole S3**, et s'affiche de manière fluide grâce aux processus d'arrière-plan (**Tasks JavaFX**) qui empêchent l'interface de figer.

iii - Cas d'utilisation 3 : Résilience et gestion des pannes réseau

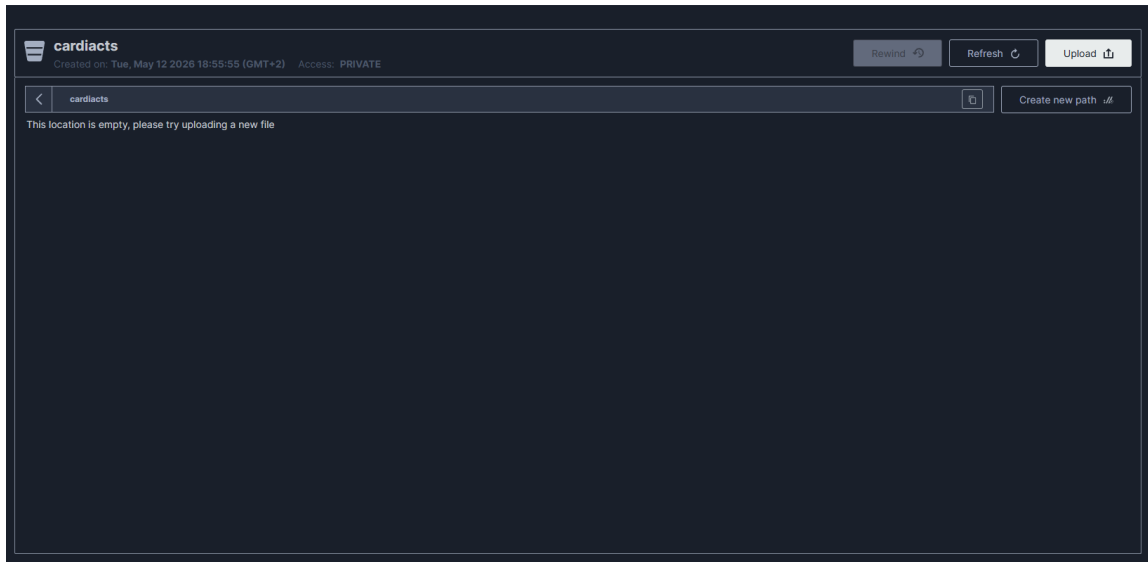
- **Entrée / Action :** Tentative d'accès aux images alors que le serveur MinIO est temporairement injoignable ou que le fichier `.env` est manquant.
- **Résultat attendu :** L'application gère l'erreur de manière robuste. Le **bloc « try/catch »** intercepte l'exception de connexion S3 et affiche une **alerte propre** à l'utilisateur, évitant le crash complet du logiciel.

Conclusions

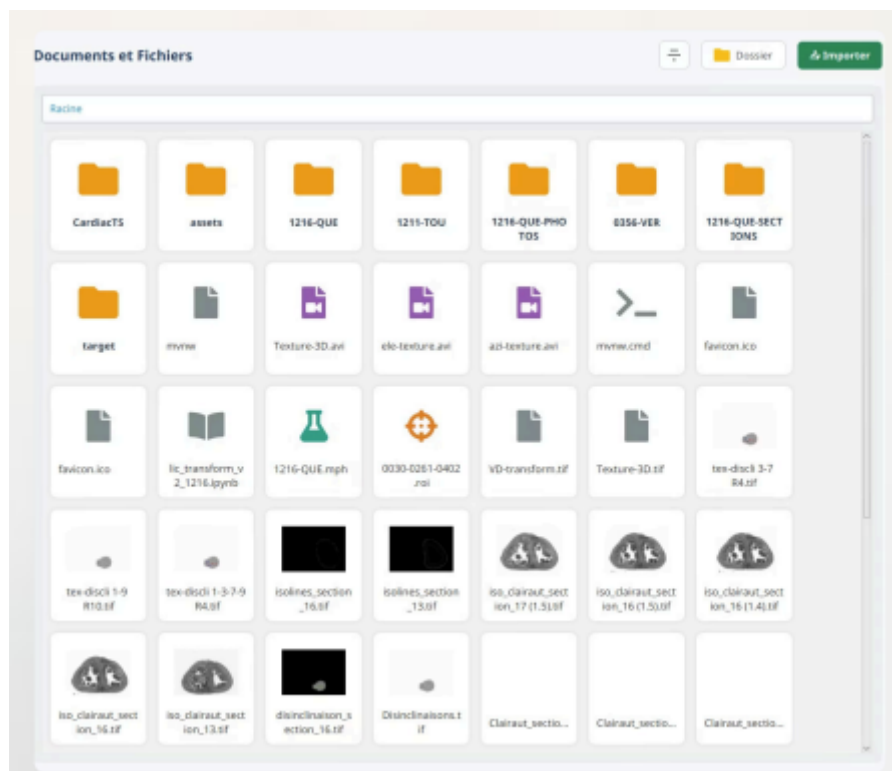
L'implémentation de cette **architecture hybride (PostgreSQL + MinIO)** a résolu définitivement les problèmes de lenteur et de sécurité du laboratoire. J'ai découvert le fonctionnement du **stockage objet**, devenu le standard incontournable des infrastructures Cloud modernes. De plus, le fait de travailler en **binôme via GitHub** m'a permis de me concentrer sur le développement de cette logique de stockage complexe, pendant que mon collègue gérait d'autres composants de l'application.

Annexes

- **Annexe 1 :** Capture de la console d'administration **MinIO** montrant l'organisation des buckets du projet CardiacTS.



- **Annexe 2 :** Capture d'écran montrant l'affichage des images dans l'interface client **JavaFX**.



Compétences associées

N°	Compétences associées
B1.1.2	Exploiter des référentiels, normes et standards adoptés (API Amazon S3)
B1.5.2	Mettre à disposition des utilisateurs un service informatique (Stockage Cloud)
B1.1.5	Gérer des sauvegardes et la persistance des données médicales
B1.4.2	Travailler en équipe informatique (Utilisation de Git/GitHub en binôme)